Autonomous Drone Detection & Response System

Aidan Malone, G00423306, 27 April 2026

Summer Presentation

The Problem of Unauthorized Drones

Addressing Security and Privacy Concerns

Drone threats are no longer limited to fixed critical infrastructure. Rapidly changing operational environments — newly secured areas, temporary installations, and forward deployments — present a growing and underserved detection challenge.

- **Temporary Events:** The 2024 Paris Olympics and 2023 UEFA Champions League Final both required drone detection capability deployed rapidly across sites with no permanent infrastructure in place.
- **Battlefield Deployments:** The conflict in Ukraine has demonstrated consumer drones being used for surveillance and attack over newly secured territory where lightweight, rapidly deployable detection is essential.

Importance of Autonomous Detection

Addressing Drone Security in Resource-Constrained Deployments

- Commercial counter-drone systems such as a **drone sentry** are designed for defence agencies and large critical infrastructure — **they require specialist installation**, fixed power infrastructure, and procurement through defence contracts, making them entirely **impractical** for temporary or rapidly changing deployments.

- The 2018 Gatwick Airport incident grounded flights for **36 hours** and cost an estimated **£64 million**, occurring at a permanent installation with dedicated security staff. At temporary sites with no dedicated systems, the response gap is even wider.

- A system that requires only a power source and is operational within minutes directly addresses this gap**. No network, no installation team, no fixed infrastructure** — this is the deployment model this project is designed for.

Research and Requirements

Key Findings and System Needs

Key Technologies

- YOLO11n nano – lightweight, single-pass detection.
- ONNX runtime – CPU optimized inference
- Kalman filter – state estimation and tracking
- OpenCV – Frame capture and annotation
- gpiozero – GPIO servo and buzzer control.

Known Limitations

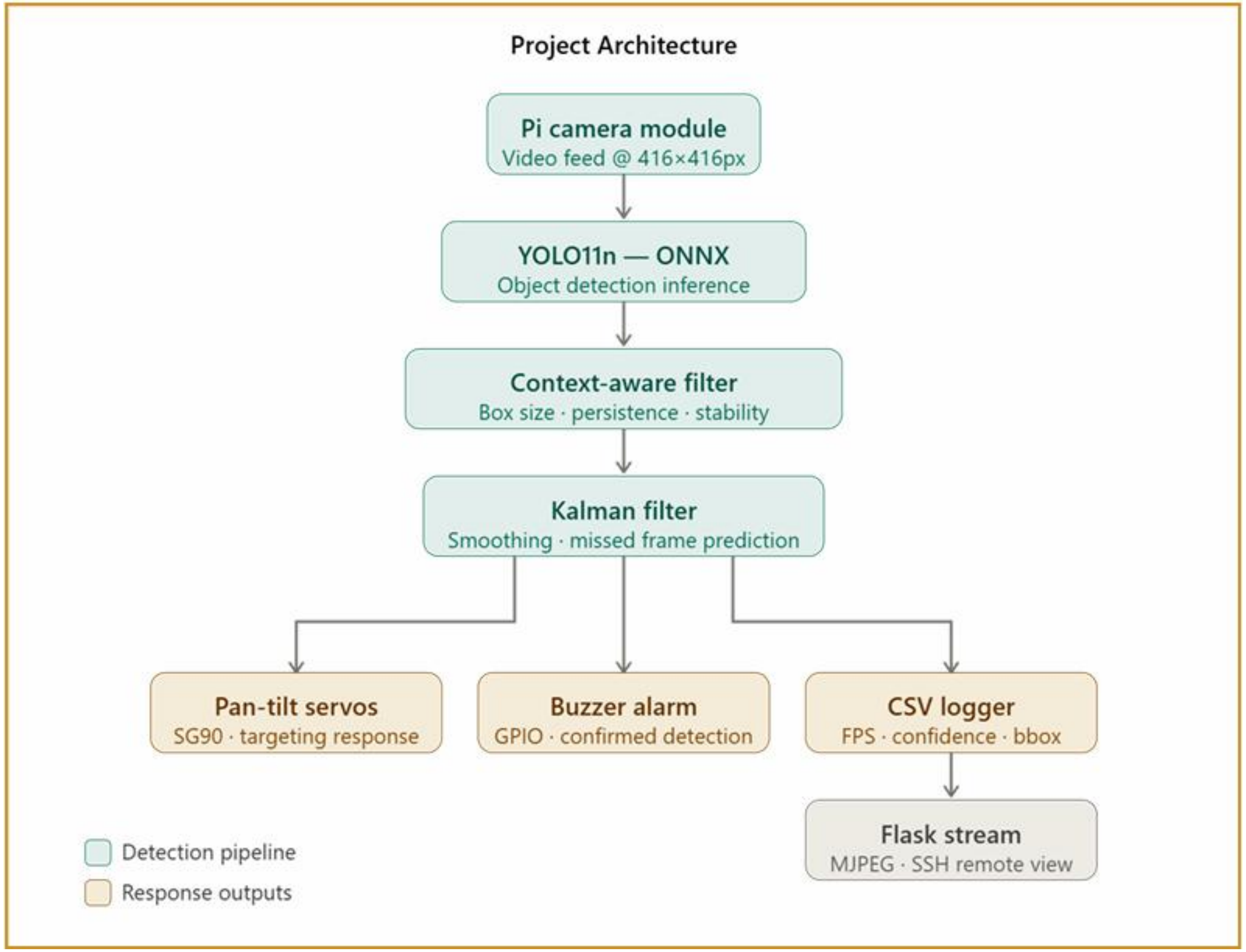
- **416px** resolution limits detection beyond 15m
- Single object tracking only
- Servo PWM interferes with buzzer signal
- Confidence threshold requires per-environment calibration

Functional Requirements Met

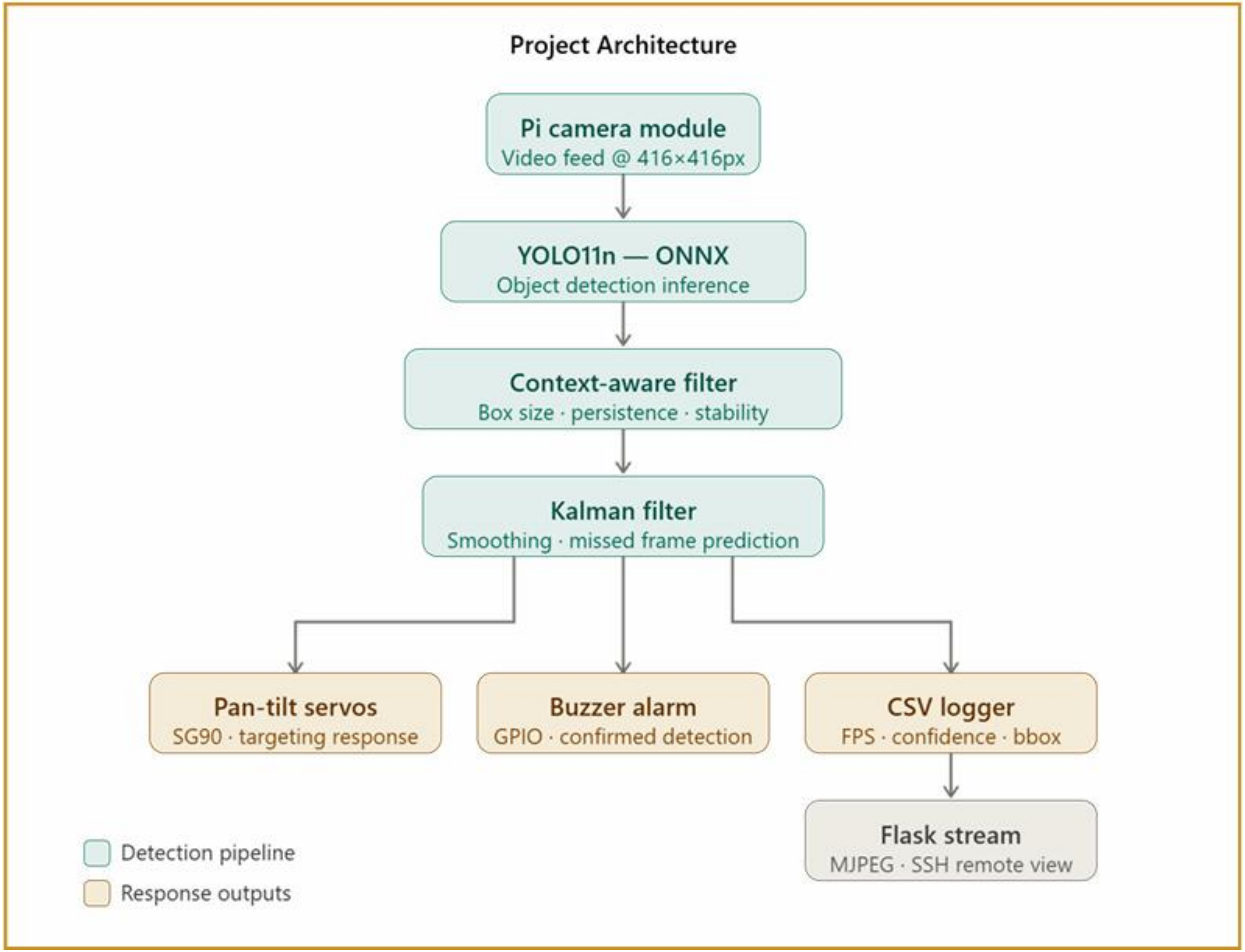
- Real-time detection at **~4 FPS** on PI 4
- **Zero False Positives in outdoor testing**
- Pan-tilt servo targeting on confirmed detection
- Audible alarm response.
- Full CSV telemetry logging per frame

Architecture and Design

- Pipeline: Camera → YOLO11n ONNX → Context Filter → Kalman Filter → [Servo / Buzzer / CSV / Flask]
- YOLO11n selected: Nano model viable at real-time on Pi 4 CPU
- ONNX over PyTorch: 2x FPS optimisation
- 416px not 640px: 2x FPS improvement, acceptable range trade-off
- Kalman filter: smooths jitter, bridges missed frames with prediction



Current Architecture



Problem Solving

ONNX Version Mismatch

- Model exported from Colab wouldn't run on the Pi's ONNX Runtime due to an opset version incompatibility.

Solution

- Re-exported with an explicitly lower opset version to match the Pi.

Bounding Box Format in CSV Logging

- YOLO returns bounding boxes as PyTorch tensors. Written directly to CSV, entries appeared as **tensor(0.45)** rather than plain numbers, breaking the metrics script.

Solution

- Added **.tolist()** on the bounding box output before logging.

Confidence Threshold Calibration

- Batch 3 first stationary test had a 98% missed detection rate despite the drone being visible — threshold too conservative for the conditions.

Solution

- Lowered base threshold mid-session. Detection rate jumped from 2.73 to 130.43 confirmed detections per minute.

Project Management

Project Management Tools - Jira

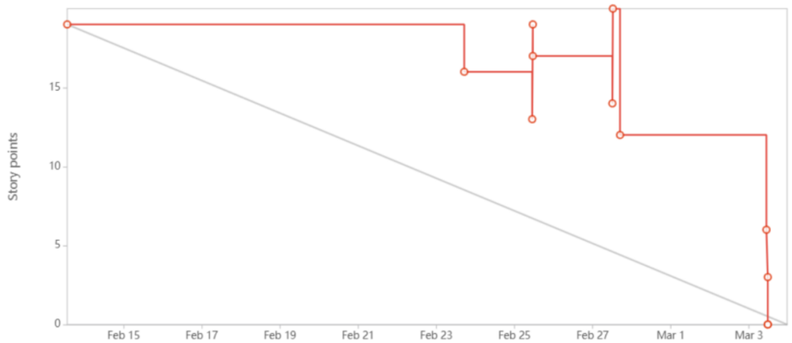
- 7 sprints across Semester 1 and 2, story points corrected from Sprint 2
- OneNote weekly logs: documented blockers, solutions, test results every sprint
- GitHub repository initialised Week 1 Semester 2 — steady incremental commits
- Attended near all weekly stand-ups, consistent updates and blockers documented
- Active collaboration with peers — shared tips and problem-solving

Link - Jira Workspace

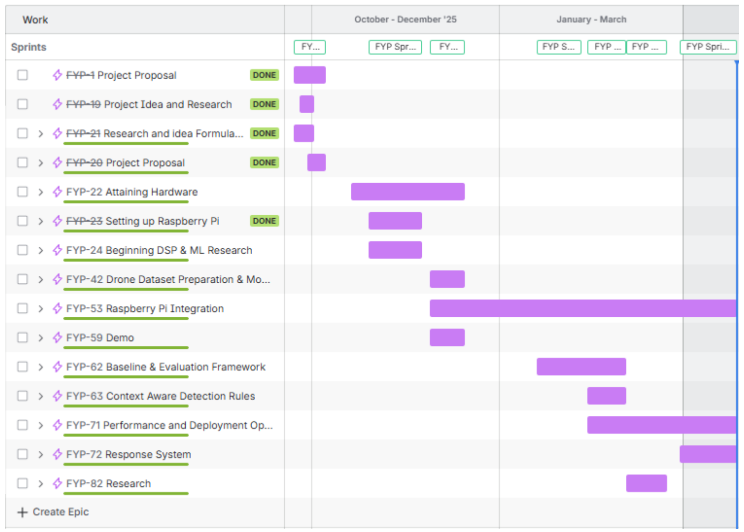
https://finalyearprojectam.atlassian.net/jira/software/projects/FYP/boards/1?atlOrigin=eyJpIjoiMmUyOTc5NTNjYjg5NDYxMWJmNTc2NmU3Y2QzNDBkNGEiLCJwIjoiaiJ9

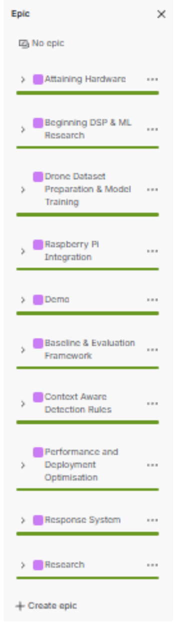
Sprint 7 – Burndown Report

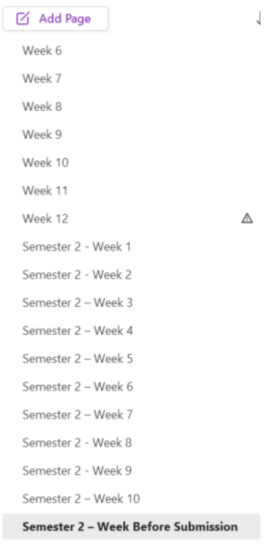
Sprint 5 – Burndown Report



Jira – Project Timeline







Continued

Teamwork Collaboration

I am always in contact with my peers, with each of us equally supporting each other with tips and knowledge. I have attached some of the emails where we have been helping each other and problem solving together.

Semester 2 – Week 7

Tuesday, March 3, 2026 9:31 AM

Search Notebooks

Add Page

- **Nasir :**
  Last week: Continued to work on state machine, implemented behavior model. Worked on connecting behavior model output to mongodb, started a new sprint.
  This week: continue work on the website, optimize frame processing performance, add behavior history tracking.
  Blocker: none

- **Jamie :**

  Last week: Integrated the metadata into the operator UI. Reviewed project progress so far with a TK Engineer, gathered recommendations, decided on new features to add over the next two sprints.

  This week: Meeting with Trane Technologies team in the USA to plan implementing my project into a manufacturing site. Meeting on Wednesday with a member of the Fluke team to review the leak coordinate system and the potentialities. Start my new 2 week sprint focusing on reading a QR code from image bytes.

  Blockers: Limited documentation/online references about the Fluke SV600 coordinates being returned from the endpoint, pausing further work on coordinate handling until after the Fluke meeting on Wednesday.

- **Pero :**

NATHAN FERRY - STUDENT
To: AIDAN MALONE - STUDENT
Mon 19 Jan 2026 2:36 PM

You replied on Tue 20 Jan 2026 1:29 PM

Hi Aidan,

I am planning on researching YOLO models for human detection for my own project and have heard that you have done similar research for your own. Would you be able to point me in the right direction re: where to start with docs etc? It would be a massive help if you could also help me with common pitfalls to avoid or lessons you learned. Thanks for the help.

Kind regards,
Nathan Ferry

09/03 14:04

Hi nathan, what exactly are you doing with your servo motors, thinking of implementing hardware into my project

NATHAN FERRY - STUDENT 09/03 14:13

So basically, I'm in stm and rpi so I have sample code in python if you need

The python code is numbers for degrees like 0 1800 3600 or something would be 0 90 etc

The stm is trickier

Basically I learned what vals the registers would have for the servos to be 0 and their max which is supposed to be 180 but i got like 220

Then with the minimum and max val i made a function that makes a register val to scale it so if the max is 8000 for 220 and you want 110 it would set it to 4000

Sounds bad but its not

09/03 14:24

Nice, so ideally I would like to stick to python as my code is all stored natively on pi, I'm thinking of having a sort of laser targeter using servos, it would use the camera co-ordinates and target the centre of the detected bounding boxes. do you have anything done along those lines?

NATHAN FERRY - STUDENT 09/03 14:28

https://medium.com/@makerhacks/jitter-free-servo-control-on-the-raspberry-pi-f596bf07d09f

Drone_Detection_And_Response_System Public

Pin · Watch 0 · Fork 0 · Star 0

master · 2 Branches · 0 Tags · Go to file · Add file · Code

AidanM324 Merge pull request #33 from AidanM324/feature1 · c3cd58c · yesterday · 69 Commits

drone_yolo11n_ncnn	Made updates and retrain to models, changed formats on ca...	2 months ago
models	Updated and moved models, added new models, changed co...	last month
templates	transferred detection logic, made integrated code from a tut...	3 months ago
.gitignore	Created new Repo for recording changes and updates, comm...	3 months ago
baseline_metrics.py	Added conf threshold max and min + conf minus thres perce...	4 days ago
buzzer.py	Implemented a Passive buzzer into my detection script, fixed ...	last month
camera.py	Made updates and retrain to models, changed formats on ca...	2 months ago
data_logging.py	Made Final clean up, Also fixed conf threshold limits + GPIO i...	yesterday
detection.py	Made Final clean up, Also fixed conf threshold limits + GPIO i...	yesterday
main.py	Made Final clean up, Also fixed conf threshold limits + GPIO i...	yesterday
raw_frame_logger.py	Added raw camera frame logger for baseline frame collection...	2 months ago

About

No description, website, or topics provided.

Activity
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

AidanM324 Aidan Malone

Implementation

Context Aware Filtering Pipeline

- Raw YOLO detections pass through four stages: minimum and maximum box area thresholds, N=3 temporal persistence, and adaptive confidence thresholding. False positives reduced from 58.2 per minute to near zero outdoors.

Key Result:

- Temporal persistence was the single most impactful improvement — three consecutive valid frames required before confirming any detection.

Kalman Filter Tracking

- A Kalman filter smooths the bounding box position across frames and predicts where the drone is during frames where YOLO misses it. This keeps the tracking stable at low frame rates.

Key Result:

- Eliminates bounding box jitter and maintains servo targeting through brief detection dropouts.

Pan-Tilt Servo System

- Two SG90 servos powered from a dedicated USB cable. Kalman-smoothed centre mapped to servo range with exponential smoothing α=0.2 to reduce jitter.

Key Result:

- USB power avoids GPIO instability. PWM buzzer interference is a known limitation — a dedicated PWM controller is the planned fix.

Testing and Results

Three Batches of Outdoor Testing – Real CSV Log Data

	Batch 1 – Indoor Baseline	Batch 2 – Outdoor	Batch 3 – Full System
Date	Semester 1	March 2026	April 2026
Avg FPS	~1-2	3.46-3.92	3.57-3.78
FP/min (No Drone)	58.2	0.00	0.00
Det/min (Stationary)	N/A	69.78	130.43
Missed (Stationary)	N/A	64.7%	38.0%
Missed (Moving)	N/A	82.1%	62.0%
Servo and Buzzer	No	No	Yes

Batch 1:
High indoor FP rate confirmed need for filtering pipeline. ONNX provided 2x FPS over PyTorch.

Batch 2:
Zero false positives outdoors. Missed detections at distance identified as remaining limitation.

Batch 3:
Confidence threshold calibration critical. Detection rate increased greatly vs Batch 2 after adjustment.

Future Work

Hardware Upgrade

Raspberry Pi 5 would increase FPS well beyond 4, enabling higher resolution and longer detection range without changing the model.

Key point: The full pipeline is already built — hardware is the only bottleneck.

Acoustic Sensor Fusion

USB microphone and DSP pipeline to detect drone rotor noise. Extends detection beyond visual limits, particularly useful for dark drones. Originally planned in the project scope.

Key point: The most meaningful capability extension for the next iteration.

Wider System Integration

This detection project could slot into a broader layered counter-drone architecture combining radar, RF scanning, and acoustic sensing — acting as the close-range confirmation and response component.

Key point: The modular codebase allows new sensor inputs without changes to the core pipeline.

Links

25-26 6802 -- PROJECT ENGINEERING Notebook

Protect Your Sky: A Survey of Counter Unmanned Aerial Vehicle Systems | IEEE Journals & Magazine | IEEE Xplore

Small Drones, Big Problems: Managing the Unmanned Threat to the Homeland - Foreign Policy Research Institute

Airport Drone Detection 2025: How to Choose, Compare & Comply

What Is Drone Detection and Why Is It Important in 2025

GitHub: https://github.com/AidanM324/Drone_Detection_And_Response_System.git

Drone Dataset (UAV) - Dataset Ninja

https://docs.ultralytics.com